

JobPilot: Smart Job Matcher

BAX-423 Final Project Technical Brief

GitHub: <https://github.com/yoyo2809/jobpilot>

Deployment: Streamlit Cloud demo submitted with the final project materials.

Page 1: Product Goal, Data, And System Architecture

Product Goal, Data, And System Architecture

JobPilot is an end-to-end job recommendation and resume tailoring application for students and career switchers. The core problem is that generic job boards return too many broad keyword matches, while real users have strict constraints: salary, location, visa needs, seniority, industry exclusions, and career-transition goals. JobPilot turns those constraints into a repeatable ranking pipeline that can be tested against four required personas.

The app uses a 30,000-posting offline LinkedIn/Kaggle snapshot plus optional live Adzuna ingestion. The offline artifacts are included so the demo can run even if API keys are unavailable:

- data/jobs_snapshot.zip: source snapshot used for rebuilds and audit.
- data/jobpilot.db.zip: SQLite database used by the Streamlit app.
- data/faiss_index.bin: vector index for dense retrieval.
- data/index_metadata.pkl: job-id metadata aligned with FAISS vectors.

The running Streamlit flow is:

Resume/manual profile -> query construction -> FAISS recall -> hard filters
-> weighted re-ranking -> explanation panel -> feedback learning -> resume export

Users may upload a PDF/DOCX resume or manually enter background, job preferences, target roles, skills, location, salary, visa sponsorship needs, and dealbreakers. Manual fields are important for the required test personas because they allow deterministic reproduction without needing separate resume files. Uploaded resumes are parsed with PyPDF2/python-docx and Gemini, then merged with manual preferences.

The sidebar Fetch New Jobs control calls Adzuna, normalizes job JSON into the same schema, deduplicates against SQLite, inserts only new jobs, and appends new vectors to the FAISS index. This satisfies the streaming/new-data requirement while preserving a stable offline demo.

JobPilot: Smart Job Matcher

BAX-423 Final Project Technical Brief

Page 2: BAX-423 Techniques And Ranking Logic

BAX-423 Techniques And Ranking Logic

JobPilot uses three course-relevant technical layers rather than a single keyword search.

Technique 1: Dense vector retrieval. Job titles, skills, and descriptions are embedded with sentence-transformers/all-MiniLM-L6-v2. FAISS retrieves semantic candidates, so related terms such as "MLOps", "ML platform", "PyTorch", and "deep learning" can match even when exact words differ. This is the Lecture 5-style embedding retrieval layer.

Technique 2: Multi-stage recommendation. The system separates recall, hard filtering, and re-ranking. This matters because embeddings alone may retrieve semantically similar but unacceptable jobs, such as senior roles for a junior candidate or defense companies for a persona that excludes defense. Hard filters enforce dealbreakers before ranking. The app filters seniority, contract/temporary roles, defense/military terms, salary floor, visa-negative language, junior/entry-level exclusions, tiny-startup proxies, and persona-specific role relevance.

If FAISS recall becomes too narrow after strict filters, JobPilot searches the full offline snapshot as a fallback while applying the same hard filters. This prevents the UI from showing only four jobs when the pass criteria require Top-10 evaluation.

The final score is transparent:

$$\text{Score} = 0.25 * \text{Semantic} + 0.25 * \text{Skill} + 0.25 * \text{Role} + 0.10 * \text{Location} + 0.15 * \text{Feedback}$$

Technique 3: Adaptive feedback. Like/pass/skip events are stored in SQLite. Accept adds positive weight, reject adds stronger negative weight, and skip is neutral. Feedback propagates to exact job IDs, companies, titles, and experience levels. For the Persona 4 requirement, the app includes a Simulate small-company rejection button so the demo can prove that startup-like companies are down-weighted even if no small company appears in the visible Top-10.

Technique 4: Generative AI resume tailoring. Gemini generates a tailored Markdown resume from the selected job, candidate profile, target roles, dealbreakers, and inferred pass criteria. The prompt instructs the model to avoid violating persona constraints. For example, ML pivots emphasize Python/modeling instead of Excel/reporting, infrastructure candidates frame Kafka/Spark/Kubernetes as ML platform skills, and research candidates lead with publications.

JobPilot: Smart Job Matcher

BAX-423 Final Project Technical Brief

Page 3: Required Persona Evaluation

Required Persona Evaluation

The Benchmark tab compares BM25 keyword retrieval, embedding-only FAISS, and the full multi-stage system with Precision@10 and NDCG@10. It also performs deterministic pass/fail checks for the required personas. The app UI shows Top-10 results, not Top-20, so screenshots and metrics align with the rubric.

Persona 1: Aisha, ML career pivoter.

Input: Data Analyst with three years of retail experience, wants ML Engineering, no production ML experience, Bay Area, salary at least \$140,000, excludes Senior/Staff/Defense/Military and roles requiring 5+ years of ML experience.

Validation: Top-10 must have zero Senior/Staff/Principal/Lead/Manager/L5/L6/Defense/Military signals. Every row must have a visibly ML/AI-focused title such as Machine Learning Engineer, Applied Scientist, NLP Engineer, or AI Research Scientist. Resume generation highlights Python, scikit-learn, PyTorch, modeling, and ML project framing rather than Excel/reporting.

Status: PASS.

Persona 2: Marcus, new graduate.

Input: Recent MSBA graduate with internships, no full-time experience, prefers tech or healthcare, target roles include Data Analyst, BI Analyst, and Junior Data Scientist, excludes 3+ years, 5+ years, contract, and unpaid work.

Validation: Results are not allowed to remain stuck on Aisha's ML-only query. Changing preferences invalidates cached rankings and re-runs the pipeline. Top-10 must avoid senior, contract, unpaid, and high-experience wording while returning Data Analyst/BI/junior-oriented roles.

Status: PASS.

Persona 3: Priya, experienced niche candidate.

Input: Senior Software Engineer with seven years in fintech, wants ML/AI infrastructure, prefers NYC or remote and companies with 100+ employees, excludes Junior/Entry roles.

Validation: Top-10 has zero Junior/Entry roles and zero tiny-startup proxy terms such as seed, stealth, pre-seed, early-stage, small startup, or startup. Why-ranked explanations show the no-junior and no-tiny-startup checks. Resume generation labels Kafka, Spark, Kubernetes, AWS, TensorFlow, and MLOps as ML infrastructure/platform skills rather than generic backend tools.

Status: PASS.

Persona 4: Kenji, international research candidate.

Input: International MSCS student on OPT, needs H-1B sponsorship within one year, has published research, wants Research Scientist or ML Engineer roles, excludes contract, 1099, and temporary roles.

Validation: Top-10 has zero contract/temp/1099 and no explicit no-sponsorship language. Ranking favors known large employers, likely visa sponsors, universities, and research labs using transparent proxy terms because the dataset lacks guaranteed sponsorship fields. Resume generation leads with publications and research output. Simulated small-company rejection demonstrates that learning deprioritizes startup-like companies after rejection.

Status: PASS.

JobPilot: Smart Job Matcher

BAX-423 Final Project Technical Brief

Page 4: Demo Clarity, Deliverables, And Limitations

Demo Clarity, Deliverables, And Limitations

The demo is designed to be clear in five steps:

1. Open the Streamlit app and enter one persona's background, job preferences, target roles, skills, location, salary, visa toggle, and dealbreakers.
2. Click Find Matches and review exactly Top-10 ranked jobs.
3. Open Why ranked here? to inspect score components and persona pass criteria checks.
4. Like/pass/skip a job, or use the small-company rejection simulation for Persona 4, then rerun ranking to show adaptive learning.
5. Click Generate Resume to produce a persona-aware Markdown resume for a selected job, then download jobs or resume outputs.

The submitted files include:

- code/: Streamlit app, ranking engine, ingestion, embeddings, benchmark, feedback, and resume generation modules.
- data/: offline database/index artifacts required for a reliable demo.
- brief.pdf: this four-page technical brief.
- prompts.md: major LLM and design prompts used during development.
- BAX423_Final_Project_Submission.zip: GitHub-friendly submission package.

Important limitations are intentionally surfaced in the UI and brief. First, company size is not a native field in the dataset. The "no companies with <100 employees" criterion is implemented through transparent tiny-startup proxies, not true LinkedIn company-size scraping. Second, H-1B sponsorship is also a proxy based on company/research-lab terms and explicit negative language such as "no sponsorship"; production use should enrich jobs with H-1B disclosure or firmographic APIs. Third, seniority and experience filters are text heuristics, so unusual wording may require manual review. Fourth, generated resumes should be reviewed by a human before use because sparse persona inputs can cause the LLM to over-expand experience.

Despite these limitations, JobPilot meets the project goals: it integrates a large offline dataset with optional live ingestion, applies embedding retrieval and multi-stage ranking, provides explainable Top-10 recommendations, learns from feedback, benchmarks ranking approaches, and generates resumes that follow the persona-specific pass criteria.